mobile-adapter

Mobile Adapter

The mobile adapter provides an interface for testing applications on mobile devices. It abstracts device interaction (installation, launching, input, screenshots) behind a common contract.

MobileAdapter Interface

Defined in `adapter_mobile.go`:

```
type MobileAdapter interface {
    IsDeviceAvailable(ctx context.Context) (bool, error)
    InstallApp(ctx context.Context, appPath string) error
    LaunchApp(ctx context.Context) error
    StopApp(ctx context.Context) error
    IsAppRunning(ctx context.Context) (bool, error)
    TakeScreenshot(ctx context.Context) ([]byte, error)
    Tap(ctx context.Context, x, y int) error
    SendKeys(ctx context.Context, text string) error
    PressKey(ctx context.Context, keycode string) error
    WaitForApp(ctx context.Context, timeout time.Duration) error
    RunInstrumentedTests(ctx context.Context, testClass string)
        (*TestResult, error)
    Close(ctx context.Context) error
    Available(ctx context.Context) bool
}
```

Method Summary

Method	Purpose
`IsDeviceAvailable`	Check if the target device is connected and ready
`InstallApp`	Install an application binary (APK, AAB, or IPA) onto the device
`LaunchApp`	Start the configured application
`StopApp`	Force-stop the running application

Method	Purpose
`IsAppRunning`	Check if the application process is active
`TakeScreenshot`	Capture the device screen as a PNG byte slice
`Tap`	Perform a tap gesture at screen coordinates
`SendKeys`	Type text into the currently focused input
`PressKey`	Send a key event (e.g., `KEYCODE_BACK`, `KEYCODE_HOME`)
`WaitForApp`	Block until the app is running or timeout expires
`RunInstrumentedTests`	Run on-device instrumented tests, optionally filtered by class
`Close`	Disconnect from the device and release resources
`Available`	Check if the automation tool is installed

Configuration Type

MobileConfig

```
type MobileConfig struct {
    PackageName  string `json:"package_name"`   // e.g.,
        "com.example.app"
    ActivityName string `json:"activity_name"`  // e.g.,
        ".MainActivity"
    DeviceSerial string `json:"device_serial"`  // optional;
        targets specific device
}
```

- `PackageName` is required for launching, stopping, and checking if the app is running.
- `ActivityName` is used with `PackageName` to form the component name for `am start`.
- `DeviceSerial` is optional. When set, all ADB commands include `-s <serial>` to target a specific device. When empty, ADB uses the default connected device.

ADBCLIAdapter

The built-in implementation shells out to the `adb` (Android Debug Bridge) command-line tool for all device interactions.

Constructor

```
config := userflow.MobileConfig{
    PackageName:  "com.example.myapp",
    ActivityName: ".MainActivity",
    DeviceSerial: "emulator-5554", // optional
}
adapter := userflow.NewADBCLIAdapter(config)
```

How It Works

Each method constructs an `adb` command with the appropriate arguments. If `DeviceSerial` is configured, `-s <serial>` is prepended to every command.

Device detection: Parses `adb devices` output, looking for lines where the second field is `"device"`.

App launch: Executes `adb shell am start -n <package>/<activity>`.

App stop: Executes `adb shell am force-stop <package>`.

App running check: Executes `adb shell pidof <package>`. Returns true if the output is non-empty.

Screenshots: Three-step process:

1. `adb shell screencap -p /sdcard/screenshot.png` -- capture on device.
2. `adb pull /sdcard/screenshot.png /tmp/adb-screenshot-<timestamp>.png` -- pull to host.
3. Read the local file, then clean up both the device and host copies.

Input: Uses `adb shell input tap <x> <y>`, `adb shell input text <text>`, and `adb shell input keyevent <keycode>`.

WaitForApp: Polls `IsAppRunning` every 500ms until the app is detected or the timeout expires.

Instrumented tests: Runs `adb shell am instrument -w` with the AndroidJUnitRunner. Optionally filters by test class with `-e class <name>`. Parses the output to extract test counts.

Availability

`Available()` checks if the `adb` binary exists in PATH via `exec.LookPath("adb")`.

Cleanup

`Close()` is a no-op for ADB since the device connection does not require explicit cleanup.

Example: Mobile Launch Challenge

```
adapter := userflow.NewADBCLIAdapter(userflow.MobileConfig{
    PackageName:  "com.example.myapp",
    ActivityName: ".MainActivity",
})

challenge := userflow.NewMobileLaunchChallenge(
    "CH-MOBILE-001",
    "App Launch and Stability",
    "Install, launch, and verify the app remains stable for 10
        seconds",
    nil,
    adapter,
    "/path/to/app-debug.apk",
    10*time.Second,
)
```

Example: Mobile Flow

```
flow := userflow.MobileFlow{
    Name: "onboarding-flow",
    Config: userflow.MobileConfig{
        PackageName:  "com.example.myapp",
        ActivityName: ".MainActivity",
    },
    Steps: []userflow.MobileStep{
        {Name: "launch", Action: "launch"},
        {Name: "tap-next", Action: "tap", X: 540, Y: 1800},
        {Name: "type-name", Action: "send_keys", Value: "John"},
        {Name: "press-enter", Action: "press_key", Value:
        "KEYCODE_ENTER"},
        {Name: "capture", Action: "screenshot"},
        {Name: "verify-running", Action: "assert_running"},
    },
}

challenge := userflow.NewMobileFlowChallenge(
    "CH-MOBILE-002",
    "Onboarding Flow",
    "Walk through the onboarding screens",
    []challenge.ID{"CH-MOBILE-001"},
    adapter,
```

```
    flow,
)
```

Supported Mobile Actions

Action	Fields Used	Behavior
`launch`	--	Launches the configured app
`stop`	--	Force-stops the app
`tap`	`X`, `Y`	Taps at screen coordinates
`send_keys`	`Value`	Types text into focused input
`press_key`	`Value`	Sends a key event
`wait`	--	Waits for app (5-second timeout)
`screenshot`	--	Captures device screen
`assert_running`	--	Fails if app is not running